

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 2: La orientación a objetos

Principios de la orientación a objetos. Comparación con la programación estructurada

Clases de objetos

Objetos

Principios de la orientación a objetos. Comparación con la programación estructurada

1. Ocultación de información (information hiding)

El principio de ocultación de información busca restringir el acceso a los detalles internos de un objeto, exponiendo solo lo necesario a otras partes del código. Esto mejora la seguridad y facilita el mantenimiento.

Ejemplo en PHP:

```
<?php
class Usuario {
    private $password; // Oculto al exterior

    public function __construct($password) {
        $this->password = $password;
    }

    public function verificarPassword($input) {
        return $this->password === $input;
    }
}

$usuario = new Usuario("secreto123");
echo $usuario->verificarPassword("secreto123") ? "Acceso permitido" : "Acceso denegado";
?>
```

◆ Aquí, la propiedad `$password` es privada, evitando que pueda ser accedida o modificada directamente.

2. El tipo abstracto de datos (ADT). Encapsulado de datos

Un **Tipo Abstracto de Datos (ADT)** define una estructura con operaciones permitidas, sin revelar su implementación interna. El **encapsulamiento** es la técnica que impide el acceso directo a los datos internos del objeto.

Ejemplo en PHP:

```
class CuentaBancaria {  
    private $saldo;  
  
    public function __construct($saldoInicial) {  
        $this->saldo = $saldoInicial;  
    }  
  
    public function depositar($cantidad) {  
        if ($cantidad > 0) {  
            $this->saldo += $cantidad;  
        }  
    }  
  
    public function obtenerSaldo() {  
        return $this->saldo;  
    }  
}  
  
$cuenta = new CuentaBancaria(100);  
$cuenta->depositar(50);  
echo "Saldo: " . $cuenta->obtenerSaldo();
```

◆ Aquí, el saldo está protegido y solo puede modificarse a través de métodos específicos.

3. Paso de mensajes

El paso de mensajes en POO se refiere a la comunicación entre objetos a través de la invocación de métodos. En PHP, esto se logra llamando a métodos de una clase desde otra.

Ejemplo en PHP:

```
class Motor
{
    public function encender()
    {
        return "Motor encendido";
    }
}

class Coche
{
    private $motor;

    public function __construct()
    {
        $this->motor = new Motor();
    }

    public function arrancar()
    {
        return $this->motor->encender();
    }
}

$coche = new Coche();
echo $coche->arrancar();
```

◆ El objeto Coche envía un mensaje al objeto Motor pidiéndole que se encienda.

Clases de objetos

Las clases son plantillas para crear objetos. Definen atributos (datos) y métodos (funcionalidades) que los objetos de la misma clase compartirán.

Tomemos como base nuestro ejemplo anterior de un Coche y expandámoslo con los siguientes conceptos:

1. Atributos, variables de estado y variables de clase

- **Atributos:** Son las propiedades de un objeto (ej. color, marca, velocidad).
- **Variables de estado:** Representan el estado actual de un objeto (ej. si el coche está encendido o apagado).
- **Variables de clase:** Son compartidas por todos los objetos de la clase y se declaran como static.

Ejemplo:

```
class Coche
{
    private $marca;
    private $modelo;
    private $velocidad;
    private $encendido = false;
    private static $numCoches = 0; // Variable de clase

    public function __construct($marca, $modelo)
    {
        $this->marca = $marca;
        $this->modelo = $modelo;
        $this->velocidad = 0;
        self::$numCoches++; // Incrementa el número de coches creados
    }

    public function encender()
    {
```

```

        $this->encendido = true;
        return "El coche {$this->marca} {$this->modelo} está encendido.";
    }

    public function acelerar($incremento)
    {
        if ($this->encendido) {
            $this->velocidad += $incremento;
            return "Velocidad actual: {$this->velocidad} km/h.";
        } else {
            return "No puedes acelerar un coche apagado.";
        }
    }

    public static function totalCoches()
    {
        return "Se han creado " . self::$numCoches . " coches.";
    }
}

// Crear objetos
$coche1 = new Coche("Toyota", "Corolla");
$coche2 = new Coche("Honda", "Civic");

echo $coche1->encender();
echo "<br>";
echo $coche1->acelerar(30);
echo "<br>";
echo Coche::totalCoches(); // Llamando a un método de clase

```

◆ Aquí vemos atributos (marca, modelo, velocidad), una variable de clase (\$numCoches) y el estado del coche (encendido).

2. Métodos. Requisitos e invariantes

Los **métodos** son funciones dentro de la clase que permiten manipular los datos y realizar acciones.

Los **requisitos** son condiciones previas necesarias para ejecutar un método.

Los **invariantes** son condiciones que deben cumplirse en todo momento.

Ejemplo con requisitos e invariantes:

```
class Coche
{
    private $marca;
    private $modelo;
    private $velocidad = 0;
    private $encendido = false;

    public function __construct($marca, $modelo)
    {
        $this->marca = $marca;
        $this->modelo = $modelo;
    }

    public function encender()
    {
        if (!$this->encendido) { // Requisito: el coche debe estar apagado
            $this->encendido = true;
            return "El coche se ha encendido.";
        }
        return "El coche ya estaba encendido.";
    }

    public function acelerar($incremento)
    {
        if (!$this->encendido) {
            return "No puedes acelerar un coche apagado.";
        }
    }
}
```

```

    }

    $this->velocidad += $incremento;
    if ($this->velocidad > 200) { // Invariante: la velocidad no puede superar 200 km/h
        $this->velocidad = 200;
    }
    return "Velocidad actual: {$this->velocidad} km/h.";
}
}
$coche = new CocheB("Toyota", "Corolla");

echo $coche->encender();
echo "<br>";
echo $coche->acelerar(50);
echo "<br>";
echo $coche->acelerar(180);

```

◆ Aquí, el coche solo puede acelerar si está encendido y la velocidad no puede superar 200 km/h.

3. Gestión de excepciones

Las excepciones permiten manejar errores sin interrumpir la ejecución del programa. En PHP, se usa try, catch y throw.

Ejemplo de excepción al intentar acelerar un coche apagado:

```
class Coche {
    private $encendido = false;
    private $velocidad = 0;

    public function encender() {
        $this->encendido = true;
    }

    public function acelerar($incremento) {
        if (!$this->encendido) {
            throw new Exception("No puedes acelerar un coche apagado.");
        }
        $this->velocidad += $incremento;
        return "Velocidad actual: {$this->velocidad} km/h.";
    }
}

$coche = new Coche();

try {
    echo $coche->acelerar(30);
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
```

◆ Aquí usamos `throw new Exception()` para lanzar un error si intentamos acelerar un coche apagado.

4. Agregación de clases

La agregación es un tipo de relación entre clases donde una clase contiene instancias de otra clase, pero la existencia de ambas no está completamente ligada.

Ejemplo: un coche tiene un motor (agregación)

```
class Motor {
    private $potencia;

    public function __construct($potencia) {
        $this->potencia = $potencia;
    }

    public function obtenerPotencia() {
        return "{$this->potencia} HP";
    }
}

class Coche {
    private $marca;
    private $motor; // Agregación

    public function __construct($marca, Motor $motor) {
        $this->marca = $marca;
        $this->motor = $motor;
    }

    public function obtenerInfo() {
        return "Coche {$this->marca} con motor de " . $this->motor->obtenerPotencia();
    }
}
```

```
$motor = new Motor(150);  
$coche = new Coche("Ford", $motor);  
echo $coche->obtenirInfo();
```

Objetos

Los **objetos** son instancias de una clase y representan entidades con atributos y comportamientos propios. En PHP, los objetos se crean a partir de una clase utilizando la palabra clave new.

1. Creación y destrucción de objetos

Un objeto se crea con new Clase(). En PHP, se pueden definir métodos especiales como __construct() y __destruct() para inicializar y limpiar los recursos de un objeto.

Ejemplo: Creación y destrucción de un coche

```
class Coche
{
    private $marca;
    private $modelo;

    public function __construct($marca, $modelo)
    {
        $this->marca = $marca;
        $this->modelo = $modelo;
        echo "El coche {$this->marca} {$this->modelo} ha sido creado.<br>";
    }

    public function __destruct()
    {
        echo "El coche {$this->marca} {$this->modelo} ha sido destruido.<br>";
    }
}

// Crear y destruir objetos
```

```
$coche1 = new Coche("Toyota", "Corolla");  
$coche2 = new Coche("Honda", "Civic");  
  
unset($coche1); // Destruir explícitamente un objeto  
  
echo "El script continúa ejecutándose...<br>"
```

◆ *El método `__destruct()` se ejecuta automáticamente cuando el objeto deja de estar en uso.*

2. Llamada a métodos de un objeto

Para acceder a los métodos de un objeto se usa ->.

Ejemplo: Llamar a métodos de un objeto

```
class Coche {  
    private $marca;  
    private $modelo;  
  
    public function __construct($marca, $modelo) {  
        $this->marca = $marca;  
        $this->modelo = $modelo;  
    }  
  
    public function obtenerInfo() {  
        return "Coche: {$this->marca} {$this->modelo}";  
    }  
}  
  
$coche = new Coche("Ford", "Focus");  
echo $coche->obtenerInfo();
```

◆ Aquí creamos un coche y llamamos a su método `obtenerInfo()` para obtener su información.

3. Visibilidad y uso de las variables de estado

Las variables de estado de un objeto pueden ser públicas, privadas o protegidas.

- **public:** Se pueden acceder desde cualquier parte.
- **private:** Solo accesibles dentro de la clase.
- **protected:** Accesibles dentro de la clase y en clases hijas.

Ejemplo con visibilidad de variables:

```
class Coche {  
    public $marca; // Accesible desde cualquier parte  
    private $modelo; // Solo accesible dentro de la clase  
    protected $anio; // Accesible en la clase y en subclases  
  
    public function __construct($marca, $modelo, $anio) {  
        $this->marca = $marca;  
        $this->modelo = $modelo;  
        $this->anio = $anio;  
    }  
  
    public function obtenerModelo() {  
        return $this->modelo; // Se accede a una variable privada mediante un método público  
    }  
}  
  
$coche = new Coche("Toyota", "Corolla", 2022);  
echo $coche->marca; // OK  
// echo $coche->modelo; // ERROR: No accesible directamente  
echo $coche->obtenerModelo(); // Correcto
```

◆ *Las variables privadas solo pueden ser accedidas desde métodos internos.*

4. Referencias a objetos

En PHP, los objetos se pasan por referencia en lugar de por valor, lo que significa que modificar un objeto en una variable afecta a todas las referencias.

Ejemplo de referencia de objetos:

```
class Coche {  
    public $marca;  
    public function __construct($marca) {  
        $this->marca = $marca;  
    }  
}  
  
$coche1 = new Coche("Toyota");  
$coche2 = $coche1; // Ambas variables apuntan al mismo objeto  
  
$coche2->marca = "Honda"; // Cambia el valor en ambas referencias  
  
echo $coche1->marca; // "Honda"  
echo "<br>";  
echo $coche2->marca; // "Honda"
```

◆ *Aquí coche1 y coche2 apuntan al mismo objeto, por lo que modificar uno afecta al otro.*

5. Persistencia de objetos

La persistencia permite almacenar un objeto en un archivo o base de datos para su uso posterior. En PHP, se puede usar `serialize()` y `unserialize()`.

Ejemplo: Guardar y recuperar un objeto con `serialize()`

```
class Coche {
    public $marca;
    public function __construct($marca) {
        $this->marca = $marca;
    }
}

$coche = new Coche("Mazda");
$serializado = serialize($coche);
file_put_contents("coche.txt", $serializado); // Guardar en un archivo

// Recuperar objeto
$contenido = file_get_contents("coche.txt");
$cocheRestaurado = unserialize($contenido);

echo "Marca del coche restaurado: " . $cocheRestaurado->marca;
```

◆ *Aquí guardamos el objeto en un archivo y lo recuperamos más tarde.*

6. Optimización de memoria y recolección de basura (Garbage Collection)

PHP tiene un colector de basura automático que libera memoria cuando un objeto deja de estar en uso. Sin embargo, se puede optimizar la memoria destruyendo manualmente objetos con `unset()`.

Ejemplo de liberación de memoria:

```
class Coche {  
    public $marca;  
    public function __construct($marca) {  
        $this->marca = $marca;  
    }  
  
    public function __destruct() {  
        echo "Objeto {$this->marca} eliminado.<br>";  
    }  
}  
  
$coche = new Coche("Audi");  
unset($coche); // Se ejecuta __destruct() inmediatamente  
echo "Fin del script.";
```

◆ Usar `unset($objeto)` permite liberar memoria manualmente antes de que PHP lo haga automáticamente.